

Smart Contracts

בהכנת חוזה חכם בשפת Solidity

חוזה Solidity **לא יכול לקרוא events**.
כלומר events הם רק "לצד החיצוני" ולא ללוגיקה פנימית.

contract

המילה Contract היא כמו מחלקה (class), אבל:

- נשמרת על הבלוקצ'יין
- בעלת state קבוע - כלומר המידע נשמר על הבלוקצ'יין ולא נעלם בסיום התכנית או בין קריאות לפונקציות.
- כל פעולה שמתבצעת על החוזה היא transaction.

Uint256 private

ההגדרה של uint256 private ABC, אנחנו מגדירים משתני state - קצת כמו משתני private של מחלקה (שדות פרטיים) אבל שנשמרים בstorage של החוזה על הבלוקצ'יין. הם משמשים בתור מונה ל IDs - כל פעם שיוצרים Post, Community, או Comment נותנים לו מזהה ייחודי. יצרנו 3 מונים - לקהילות, פוסטים ותגובות.

Structs

כאן אנחנו מגדירים את המבנים שלנו, למשל עבור קהילה:

```
struct Community {  
  uint256 id;  
  string name;  
  address creator;  
  string metadataCID;  
  uint256 createdAt;  
  uint256 membersCount;  
  bool exists;  
}
```

למה אנחנו צריכים exists?

כי בסולידיטי אם ניגשים ל mapping שלא קיים - מקבלים struct ריק. אז צריך דגל כלשהו כדי

Mapping

בסולידיטי:

```
mapping(uint256 => Community) private communities;
```

שקול לחיפוי בcpp:

```
unordered_map<int, Community>
```

C++

אין iteration, אין size ואין keys. כלומר אי אפשר לעבור בלולאה - החוזה לא יודע איזה keys קיימים, וגם לא כמה entries יש. המיפוי בסולידיטי עובד עם פונק' Hashing שמחשבת לפי key ושולפת ערך מהstorage. כלומר:

```
value = storage[hash(key)]
```

(זה בנוי ככה בסולידיטי כי זה מאוד זול בgas, זמן חישוב ולא דורש שמירה של metadata)

- הוספנו בנוסף רשימה של כל הקהילות כי mapping לא מאפשר לנו לעבור על כולם (אין iteration אז שומרים רשימה נפרדת)

```
uint256[] private allCommunityIds;
```

Relations

לכל קהילה יש רשימת Post IDs

```
mapping(uint256 => uint256[]) private communityPostIds;
```

לכל פוסט יש רשימת comments

```
mapping(uint256 => uint256[]) private postCommentIds;
```

Membership

מיפוי בתוך מיפוי - בדיקה האם משתמש הוא חבר בקהילה

```
mapping(uint256 => mapping(address => bool)) public isMember;
```

מניעת כפל שמות לקהילה

```
mapping(bytes32 => bool) private communityNameExists;
```

במקום לשמור string נשמור hash כי זה יותר "זול"

Custom Errors

מקביל ל exceptions אבל יותר זולים מ string.
במקום לכתוב

```
require(x, "error message");
```

עושים

```
if (!x) revert ErrorName();
```

Events

איוונטים הם לוגים (logs) שמפורסמים לבלוקצ'יין
נשמרים בבלוקצ'יין כלוגים, ניתנים לקריאה על ידי frontend, ניתנים לסינון (filtering!) וניתנים לשימוש ע"י the Graph.

ה events לא משנים state אלא רק משדרים החוצה, כדי שה frontend יוכל להאזין לזה
הגדרת איוונט נראית קצת כמו struct, כאשר סוג משתנה יכול להיות מוגדר להיות indexed על מנת שאפשר לחפש לפי השדה הזה.

- אפשר עד 3 שדות שהם indexed

איך משתמשים בזה?

```
emit CommunityCreated(
    communityId,
    msg.sender,
    name,
    metadataCID,
    block.timestamp
);
```

הfrontend משתמש בזה אחר כך:

```
JS
contract.on("PostCreated", (postId, communityId, author) => {
    console.log("New post!", postId);
});
```

Modifiers

ה - modifier הוא מנגנון של סולידיטי שמאפשר להגדיר בדיקה או התנהגות שחוזרת על עצמה ואז "להלביש אותה" על פונקציות. שקול לפונק' חיצונית שמבצעת בדיקה לפני תחילת פונק' (למשל בדיקת גבולות מערך) מונע חזרתיות של בדיקות דומות בכל פונק'

תחביר:

```
modifier onlyOwner() {
    if (msg.sender != owner) {
        revert NotOwner();
    }
    _;
}
```

משתמשים בו ככה:

```
function doSomething() external onlyOwner {
    ...
}
```

כלומר: כשמישהו קורא לdoSomething
נכנס קודם הקוד של onlyOwner
אם הבדיקה נכשלת, אז revert

אם עוברת אז עוברים לסימון
במקום נכנסת הפונק' המקורית

הסימון _ הוא כמו להגיד "כאן תיכנס הפונק' שעליה שמנו modifier".

Write Functions

הפונקציה ליצירת קהילה חדשה:

בודקת שהשם תקין (לא ריק), שיש metadataCID, שלא קיימת כבר קהילה עם השם הזה.
ואז מייצרת ID חדש, שומרת את הקהילה, מוסיפה את היוצר כחבר קהילה, מעדכנת מבני עזר ומשדרת event.

הפונקציה להצטרפות לקהילה מצרפת את שולח הבקשה לקהילה אלא אם כן הוא כבר חבר בה או שהקהילה לא קיימת

הפונקציה ליצירת פוסט - מוודאה שהקהילה קיימת ושהמשתמש שולח הבקשה חבר בה, בנוסף הפוסט חייב להכיל תוכן

הפונקציה ליצירת תגובה - יוצרת תגובה ומוודאה שיש תוכן לתגובה ושהפוסט קיים ושהמשתמש שמנסה להגיב חבר בקהילה.

מושגים חשובים:

- ה - msg.sender היא הכתובת של מי שקרא לפונקציה
- ה - block.timestamp הוא הזמן הנוכחי לפי הבלוק
- ה - external - פונקציה שנועדה לקריאה מחוץ לחוזה
- ה - celldata - קלט read-only (יותר זול)
- ה - revert - עצירה מיידיית עם שגיאה
- ה - emit - שידור של event.

Read Functions

מסומנות ב view כלומר לא משנות STATE רק מחזירות מידע.

הfrontend משתמש בהם להציג נתונים.

לא אמורות לעלות gas אם קוראים להן מבחוץ בתור קריאת read רגילה.

מחזירות data מה-storage.

(ה-view מקביל לconst בcpp)

פונק' getCommunity מקבל מזהה ומחזירה את כל המידע על הקהילה או נכשלת אם הקהילה לא קיימת.

מחזירה tuple של ערכים כמו id, name, creator, ...

ה- memory הוא עותק זמני בתוך הרצת הפונקציה
זאת לעומת storage שאלה הנתונים הקבועים שעל הבלוקצ'יין ו-celldata שזה קלט read-only
לפונקציה

יש לנו פונקציה בשם getAllCommunityIds שעושה בדיוק מה שהשם שלה מציע. אנחנו צריכים
את זה כי בסולידיטי mapping לא מאפשר לעבור על כל המפתחות שלו, ואנחנו רוצים שתהיה
לנו דרך פשוטה לבקש מהחזרה את הקהילות.

הסימון `uint256[] memory` מחזיר מערך של מספרים - נוצר עותק זמני של המערך לצורך
ההחזרה.